

IndustryGPT: Specialized LLM Bot Using Pre-Trained Models

Retail Customer Support Chatbot

Technical Documentation

Group Members

Dheeraj Patel | Mayank Gupta | Rozi Fatma | Sadhana Balnad

GitHub: <https://github.com/Dheeraj-Patel98/IndustryGPT-LLM-Bot>

Property	Value
Industry	Retail Customer Support
Fine-Tuning Method	QLoRA — TinyLlama-1.1B-Chat
Dataset Source	Bitext Customer Support Dataset (HuggingFace)
Chatbot Interface	Gradio — Public Share Link
Training Platform	Google Colab T4 GPU (Free)
LoRA Rank / Alpha	r=16, Alpha=32

1. Introduction and Problem Statement

This project, IndustryGPT, builds a domain-specific AI chatbot for the Retail Customer Support industry by fine-tuning a pre-trained Large Language Model (LLM) using parameter-efficient techniques. Rather than training a model from scratch, we use transfer learning with TinyLlama-1.1B-Chat from Hugging Face, adapting it to retail support conversations using QLoRA (Quantized Low-Rank Adaptation).

1.1 Why Retail Customer Support?

The Retail Customer Support domain was selected because it represents one of the highest-volume, most structured use cases for conversational AI. Key challenges that AI directly addresses in this space:

- Customers need instant, 24/7 responses to order status, returns, refunds, and shipping queries
- Human support teams face high volume during peak retail seasons and cannot scale cost-effectively
- Repetitive queries (order tracking, return policies) consume a disproportionate share of agent time
- Inconsistent responses across agents damage customer trust and brand reputation
- Fine-tuned models can learn company-specific policies and terminology that generic chatbots lack

1.2 What Problem Does This Project Solve?

Generic LLMs hallucinate domain-specific facts and lack structured retail knowledge. This project solves both problems by fine-tuning TinyLlama on a structured retail instruction-response dataset, teaching the model to provide clear, polite, and factual customer support answers while minimizing hallucination through low-temperature generation and repetition penalties.

2. Project Architecture

The system follows a straightforward fine-tuning and inference pipeline. Below is the complete system flow from user input to chatbot response:

System Flow: User types question → Tokenizer encodes input → Fine-tuned TinyLlama-1.1B generates response → Decoder produces text → Gradio displays answer

Component	Technology & Role
Base Model	TinyLlama/TinyLlama-1.1B-Chat-v1.0 — fine-tuned to understand retail customer support vocabulary
Quantization	4-bit NF4 (QLoRA) — compresses model from ~4.4 GB to ~1.1 GB, fits in free T4 GPU
LoRA Adapters	Rank 16, Alpha 32, target modules: q_proj, v_proj — parameter-efficient fine-tuning
Dataset	bitext/Bitext-customer-support-llm-chatbot-training-dataset — 2,000 samples used for training
Training Framework	Hugging Face Trainer API with TrainingArguments — standard, modular fine-tuning pipeline
Optimizer	paged_adamw_8bit — memory-efficient AdamW optimizer for constrained GPU environments

Chatbot UI	Gradio ChatInterface — public share link with example questions and real-time inference
Training Platform	Google Colab T4 GPU — free, 15 GB VRAM, no local hardware required

3. Dataset Collection and Preparation

The project uses a real, publicly available dataset from HuggingFace Hub — not synthetic or hardcoded data. The dataset is then preprocessed and formatted for supervised instruction fine-tuning.

3.1 HuggingFace Dataset

Property	Value
Dataset Name	bitext/Bitext-customer-support-llm-chatbot-training-dataset
Source	HuggingFace Hub (publicly available)
Domain	Retail / E-commerce customer support
Structure	Structured instruction-response pairs categorized by issue type
Training Samples Used	2,000 samples (selected via <code>.select(range(2000))</code>)
License	Open for research use
Why This Dataset?	Real-world retail support conversations, clean & well-structured, supports supervised fine-tuning

3.2 Data Formatting

Each dataset example is formatted into a consistent prompt template that clearly separates the instruction from the response. This teaches the model to distinguish question context from generated answer:

```
def format_data(example):
    return {
        "text": f"""### Instruction:
{example['instruction']}

### Response:
{example['response']}"""
    }
```

After formatting, all columns except 'text' are removed, and tokenization converts the text into numerical input IDs that the model can process. The tokenizer uses padding to max_length=512 and sets labels equal to input_ids for causal language modeling training.

4. QLoRA Fine-Tuning

Fine-tuning adapts TinyLlama-1.1B to the Retail Customer Support domain. We use QLoRA (Quantized Low-Rank Adaptation) — the most memory-efficient fine-tuning technique available — enabling training on free Colab hardware.

4.1 What is QLoRA and Why We Use It

Standard full fine-tuning of a 1.1B parameter model in 32-bit precision requires approximately 16 GB of GPU memory just for weights, leaving nothing for gradients or activations. QLoRA solves this with two innovations:

- 4-Bit NF4 Quantization: Base model weights are compressed from 32-bit floating point to 4-bit NormalFloat (NF4) format using bitsandbytes. This reduces model memory from ~4.4 GB to ~1.1 GB — a 4x reduction with minimal quality loss.
- LoRA Adapters: The base model is frozen completely. Small trainable adapter matrices are added only to q_proj and v_proj attention layers. Only a fraction of total parameters are actually updated, saving memory and speeding up training dramatically.

4.2 Base Model: TinyLlama-1.1B-Chat-v1.0

Property	Value
HuggingFace ID	TinyLlama/TinyLlama-1.1B-Chat-v1.0
Architecture	LLaMA-2 style transformer decoder
Total Parameters	1.1 billion
Pre-training Data	Massive corpus of diverse internet text
License	Apache 2.0 — free for research and commercial use
Why TinyLlama?	Small enough for free T4 GPU, strong conversational capability, supports QLoRA

4.3 Quantization Configuration

```
bnb_config = BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_compute_dtype=torch.float16,  
    bnb_4bit_use_double_quant=True,  
    bnb_4bit_quant_type="nf4"  
)
```

4.4 LoRA Configuration

Parameter	Value & Reason
LoRA Rank (r)	16 — higher rank = more capacity. Standard balance of quality vs memory for domain adaptation.
LoRA Alpha	32 — scaling factor set to 2x rank following standard practice for stable training.
Target Modules	q_proj, v_proj — key attention projection layers where domain adaptation has most impact.
LoRA Dropout	0.1 — light regularization to prevent overfitting on the 2,000 sample dataset.
Bias	none — no bias terms are trained, reduces memory footprint.
Task Type	CAUSAL_LM — autoregressive language modeling objective.

4.5 Training Configuration

Parameter	Value & Reason
Output Directory	./retail-llm — where checkpoints are saved
Per-Device Batch Size	4 — fits in 15 GB T4 VRAM alongside quantized model
Gradient Accumulation Steps	4 — simulates effective batch size of 16 without extra memory
Number of Epochs	2 — sufficient for domain adaptation on the retail dataset
Learning Rate	2e-4 — standard rate for LoRA training
Precision	FP16 — mixed precision halves memory use and speeds training
Logging Steps	50 — loss printed every 50 steps for monitoring
Save Strategy	epoch — save checkpoint at end of each epoch
Optimizer	paged_adamw_8bit — memory-efficient optimizer for quantized training
Training Samples	2,000 — selected subset to fit within Colab session time limits

5. Reducing Hallucination

One of the major challenges with large language models is hallucination — generating incorrect or fabricated information. This project implements several strategies to minimize hallucination during inference:

Strategy	Implementation	Effect
----------	----------------	--------

Low Temperature	temperature=0.3 in model.generate()	Reduces randomness — model picks higher-probability tokens, sticking closer to training distribution
Top-p Sampling	top_p=0.9	Limits token selection to the top 90% probability mass, cutting off long-tail hallucinations
Repetition Penalty	repetition_penalty=1.2	Penalizes the model for repeating tokens, preventing rambling or looping outputs
Max Token Limit	max_new_tokens=150	Prevents the model from generating extended confabulations beyond the answer
Domain Fine-Tuning	Trained on retail-specific instruction-response pairs	Grounds the model in retail vocabulary and expected answer patterns
System Prompt	You are a professional retail customer support assistant...	Primes the model to provide clear, polite, factual answers only

5.1 Inference Code

```
def chatbot_response(message, history):
    prompt = f"""You are a professional retail customer support assistant.
    Provide clear, polite and factual answers only. Do not ask product or order
    details.

    ### Instruction:
    {message}

    ### Response:
    """

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=150,
            temperature=0.3,
            top_p=0.9,
            repetition_penalty=1.2,
            do_sample=True
        )
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    response = response.split("### Response:")[-1].strip()
    return response
```

6. Google Colab Notebook — Step by Step

The entire project runs in a single Google Colab notebook. Below is what each step does and what output you see when running it:

Step	Title	What It Does	Expected Output
1	Install Dependencies	Installs: transformers, datasets, peft, bitsandbytes, accelerate, trl, gradio with compatible versions	All libraries install without errors. Takes 2-4 minutes.
2	Imports & GPU Check	Imports PyTorch, HuggingFace modules, PEFT, and verifies CUDA is active	torch.cuda.is_available() returns True. T4 GPU with 15 GB VRAM confirmed.
3	Load Dataset	Authenticates to HuggingFace Hub and downloads bitext customer support dataset	Dataset object printed with train split info and column names.
4	Format Data	Applies format_data() to structure each example as Instruction/Response prompt	Mapped dataset with single 'text' column containing formatted prompts.
5	Load TinyLlama + QLoRA	Downloads TinyLlama-1.1B in 4-bit NF4 quantization, attaches LoRA adapters	Model loads (~2.2 GB download). print_trainable_parameters() shows ~0.5-1% trainable params.
6	Tokenize Dataset	Converts text prompts to input_ids, attention_mask, labels tensors at max_length=512	Tokenized dataset with torch tensors. 2,000 samples selected.
7	Fine-Tuning	Trains model for 2 epochs on 2,000 retail samples using Trainer API	Training loss printed every 50 steps. Should decrease from ~2.5 toward ~1.5.
8	Save Model	Saves fine-tuned LoRA adapter and tokenizer to retail-chatbot-final/	retail-chatbot-final/ directory created with adapter_model.bin and tokenizer files.
9	Launch Gradio	Creates Gradio ChatInterface with example questions and launches public share URL	Public gradio.live URL printed. Chatbot accessible from any browser.

7. Gradio Chatbot Interface

The final chatbot is built with Gradio's ChatInterface and launched with a public share link accessible from any device during demonstration.

Feature	Description
Live Chat Window	Full conversation history with user messages and bot responses in alternating chat bubbles
Example Questions	'Where is my order?', 'How do I return a product?', 'Can I change my shipping address?', 'What is your refund policy?'

Domain Restriction	System prompt constrains the model to retail customer support answers only
Hallucination Controls	Low temperature (0.3), top-p (0.9), repetition penalty (1.2), max 150 new tokens
Theme	Gradio 'soft' theme for a clean, user-friendly interface
Deployment	demo.launch() with public share link via gradio.live

7.1 Retail Topics Covered

Topic	Example Questions Handled
Order Tracking	Where is my order? What is my delivery date? My order is delayed.
Returns & Refunds	How do I return a product? What is your refund policy? How long does a refund take?
Shipping	Can I change my shipping address? What are your delivery options?
Product Queries	Is this item in stock? What sizes are available?
Account Issues	How do I reset my password? How do I update my email address?
Promotions	Do you have a student discount? How do I apply a coupon code?

8. Library Reference

All libraries are installed in Step 1 with compatible versions for Google Colab. Here is why each library is needed:

Library	Purpose	Key Usage
transformers	Load and run HuggingFace models	AutoTokenizer, AutoModelForCausalLM, TrainingArguments, Trainer
datasets	Load public HuggingFace datasets	load_dataset(), dataset.map(), dataset.select()
peft	Parameter-efficient fine-tuning (LoRA/QLoRA)	LoraConfig, get_peft_model(), model.print_trainable_parameters()
bitsandbytes	4-bit and 8-bit quantization	BitsAndBytesConfig, load_in_4bit, nf4 quant type
accelerate	Optimize GPU utilization during training	device_map='auto', mixed precision support

trl	Optional training utilities for LLMs	SFTTrainer (optional alternative to Trainer)
gradio	Build and deploy chatbot web interface	gr.ChatInterface, demo.launch()
torch	Core deep learning framework (PyTorch)	CUDA detection, tensor operations, model.generate()
huggingface_hub	Authenticate to HuggingFace for dataset access	login() with HF token

9. Evaluation Criteria Coverage

The project is assessed on the following key criteria and here is how the implementation addresses each:

Criterion	How EduBot Pro Meets It
Real Dataset	Uses bitext/Bitext-customer-support-llm-chatbot-training-dataset — a genuine, publicly available HuggingFace dataset with real retail support conversations
Pre-Trained Model	TinyLlama-1.1B-Chat-v1.0 — a proper pre-trained model from HuggingFace, not trained from scratch
Parameter-Efficient Fine-Tuning	QLoRA with LoRA rank 16 — only ~0.5-1% of parameters trained using peft library
Domain-Specific Training	Model fine-tuned specifically on retail customer support instruction-response pairs
Working Chatbot Interface	Gradio ChatInterface deployed with public share link and example questions for live demonstration
Hallucination Reduction	Multiple strategies: low temperature, top-p sampling, repetition penalty, system prompt constraints, max token limits
Reproducibility	All steps run end-to-end in a single Google Colab notebook on free T4 GPU

10. Technical Notes & Troubleshooting

Issue	Cause	Fix
CUDA not available	Colab runtime not set to GPU	Runtime → Change runtime type → T4 GPU
Out of Memory (OOM)	Batch size too large or model not quantized	Reduce per_device_train_batch_size to 2, ensure BitsAndBytesConfig is applied

HuggingFace login fails	Invalid or expired HF token	Get new token from huggingface.co/settings/tokens
Gradio URL not generated	Network issue or Gradio version mismatch	Run <code>!pip install -q gradio</code> again and re-run the cell
Training loss not decreasing	Learning rate too high or data formatting issue	Verify <code>format_data()</code> output and reduce <code>learning_rate</code> to <code>1e-4</code>
Slow inference	Model on CPU instead of CUDA	Ensure inputs are moved to <code>model.device</code> before <code>generate()</code>